

Tugas Analisis Multimedia: Image (Citra Digital)

Mata Kuliah: Sistem & Teknologi Multimedia
Nama: Naufal Haris Nurkhoirulloh
NIM: 122140040

Referensi AI Gemini: <https://gemini.google.com/share/1d766017e70f>
Referensi lain : Worksheet Multimedia week 1

Saya convert pdf pakai aplikasi ke3 karena di laptop tidak support

Deskripsi Tugas

Tugas ini bertujuan untuk memahami **representasi dasar data citra digital (image)** melalui praktik langsung memuat data, visualisasi komponen warna, serta melakukan analisis spasial sederhana menggunakan berbagai teknik dasar pengolahan citra.

Anda akan bekerja dengan satu atau beberapa gambar (foto diri, objek, atau lingkungan sekitar) untuk:

- Mengamati struktur data piksel dan channel warna (RGB, Grayscale, HSV, dsb.)
- Menganalisis perbedaan hasil visualisasi antar representasi warna
- Melakukan eksplorasi sederhana terhadap transformasi citra (cropping, filtering, edge detection, dll.)
- Menyimpulkan pengaruh setiap tahap pemrosesan terhadap persepsi visual

Fokus tugas ini adalah pada **pemahaman konsep representasi spasial citra digital** dan **interpretasi hasil visualisasi**, bukan pada manipulasi kompleks atau penerapan model pembelajaran mesin.

```
# Import library yang dibutuhkan (referensi dari image processing week 1)
import numpy as np # Untuk operasi numerik dan manipulasi array
import matplotlib.pyplot as plt # Untuk visualisasi gambar
import cv2 # OpenCV: library utama untuk pengolahan citra dan aplikasi terkait
import os # Untuk operasi file dan path
import mediapipe as mp

# Cek versi library yang diinstal
print(f"Versi numpy: {np.__version__}")
print(f"Versi matplotlib: {plt.matplotlib.__version__}")
print(f"Versi OpenCV: {cv2.__version__}")

Versi numpy: 1.26.4
Versi matplotlib: 3.10.6
Versi OpenCV: 4.11.0
```

Soal 1 — Cropping dan Konversi Warna

- Ambil sebuah gambar diri Anda (*selfie*) menggunakan kamera atau smartphone.
- Lakukan **cropping secara manual** untuk menghasilkan dua potongan:
 - Cropping **kotak persegi pada area wajah**.
 - Cropping **persegi panjang pada area latar belakang**.
- Resize hasil crop menjadi **920×920 piksel**.
- Konversi gambar menjadi **grayscale** dan **HSV**, lalu tampilkan ketiganya berdampingan.
- Tambahkan **anotasi teks** berisi nama Anda di atas kepala pada gambar hasil crop.
 - Gaya teks (font, warna, posisi, ukuran, ketebalan) **dibebaskan**.
- Jelaskan efek **cropping** dan **perubahan warna** menggunakan **Markdown**.

Cropping

```
# 1. Muat Gambar
image_path = "assets_ws4/selfie1.jpg"
img = cv2.imread(image_path)

print("Ukuran gambar:", img.shape)
print("Ukuran file gambar (dalam bytes):", os.path.getsize(image_path))
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Tampilkan gambar asli (opsional, untuk membantu menentukan koordinat)
plt.figure(figsize=(7, 5))
plt.imshow(img_rgb)
```

```
plt.axis('off')
plt.title("Citra Saya")
plt.show()
```

Ukuran gambar: (1500, 1000, 3)

Ukuran file gambar (dalam bytes): 110004

Citra Saya



```
# --- Tentukan Koordinat Manual ---
```

```
# 1. Crop Persegi untuk Wajah (ukuran asli gambar saya 1500, 1000)
```

```
y1_face, y2_face = 490, 790 # (y1, y2)
```

```
x1_face, x2_face = 360, 660 # (x1, x2)
```

```
# 2. Crop Persegi Panjang untuk Background (contoh: area 150x400)
```

```
y1_bg, y2_bg = 50, 600 # (y1, y2)
```

```
x1_bg, x2_bg = 400, 650 # (x1, x2)
```

```
# Cropping
```

```
crop_wajah = img_rgb[y1_face:y2_face, x1_face:x2_face]
```

```
crop_background = img_rgb[y1_bg:y2_bg, x1_bg:x2_bg]
```

```
print("Ukuran crop wajah:", crop_wajah.shape)
```

```
print("Ukuran file crop wajah (dalam bytes):", crop_wajah.nbytes)
```

```
# Tampilkan hasil
```

```
plt.figure(figsize=(6, 4))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(crop_wajah)
```

```
plt.title(f'foreground wajah\nShape: {crop_wajah.shape}')
```

```
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(crop_background)
```

```
plt.title(f'Background bagian atas\nShape: {crop_background.shape}')
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

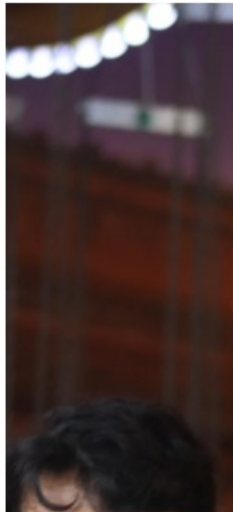
Ukuran crop wajah: (300, 300, 3)

Ukuran file crop wajah (dalam bytes): 270000

foreground wajah
Shape: (300, 300, 3)



Background bagian atas
Shape: (550, 250, 3)



```
wajah_annotated_bgr = crop_wajah.copy()

# Ambil nama dari variabel di Langkah 2
text_nama = "Naufal Haris N"

# PENGATURAN GAYA TEKS (Bebas)
font = cv2.FONT_ITALIC
font_scale = 1
font_thickness = 1
color = (255, 255, 255)
posisi = (30, 30) # (x, y)

# Tambahkan teks ke gambar BGR
cv2.putText(wajah_annotated_bgr, text_nama, posisi, font, font_scale, color, font_thickness, cv2.LINE_AA)

# Tampilkan hasil anotasi
plt.figure(figsize=(4, 4))
plt.imshow(wajah_annotated_bgr)
plt.title("Hasil Anotasi Teks")
plt.axis('off')
plt.show()
cv2.imwrite("results_ws4/wajah_annotated.jpg", cv2.cvtColor(wajah_annotated_bgr, cv2.COLOR_RGB2BGR))
```

Hasil Anotasi Teks



True

Resizing

```
# Tentukan ukuran target
target_size = (920, 920) # (width, height)

# Resize gambar wajah
resized_wajah = cv2.resize(crop_wajah, target_size)

# Resize background
resized_background = cv2.resize(crop_background, target_size)

# Tampilkan hasil
plt.figure(figsize=(6, 4))

plt.subplot(1, 2, 1)
plt.imshow(resized_wajah)
plt.title(f'foreground wajah\nShape: {resized_wajah.shape}')
plt.axis('off')

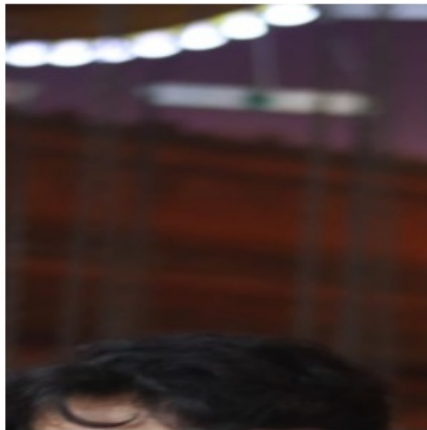
plt.subplot(1, 2, 2)
plt.imshow(resized_background)
plt.title(f'Background bagian atas\nShape: {resized_background.shape}')
plt.axis('off')

plt.tight_layout()
plt.show()
```

foreground wajah
Shape: (920, 920, 3)



Background bagian atas
Shape: (920, 920, 3)



konversi warna

```
# saya ambil yang wajah untuk di proses lebih lanjut
# Konversi Warna ke HSV
hsv_wajah = cv2.cvtColor(resized_wajah, cv2.COLOR_RGB2HSV)

# konversi warna ke greyscale
gray_wajah = cv2.cvtColor(resized_wajah, cv2.COLOR_RGB2GRAY)

plt.figure(figsize=(7, 4))

plt.subplot(1, 3, 1)
plt.imshow(resized_wajah)
plt.title(f'Wajah hasil resized')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(hsv_wajah)
plt.title(f'Wajah dalam HSV')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(gray_wajah, cmap='gray')
plt.title(f'Wajah dalam greyscale')
plt.axis('off')

plt.tight_layout()
plt.show()
cv2.imwrite("results_ws4/wajah_hsv.jpg", hsv_wajah)
cv2.imwrite("results_ws4/wajah_greyscale.jpg", gray_wajah)
```

Wajah hasil resized



Wajah dalam HSV



Wajah dalam greyscale



True

Penjelasan efek **cropping** dan **perubahan warna**

- Untuk efek cropping yaitu ukuran file mengecil karena pixel yg disimpan lebih sedikit (cropping). nah biasanya cropping untuk fokus atau ngambil area tertentu misal pada kasus ini saya mengisolasi region of interest pada area muka saja seperti itu.
- efek perubahan warna dari yg rgb (3 channel) menjadi grayscale (1 channel) gambar yang tadinya full color menjadi hanya intensitas warna keabuan. kalau file dalam jumlah besar dan banyak maka akan lebih cepat prosesnya.
- efek perubahan warna menjadi HSV berdasarkan yang saya pahami dari diskusi dengan gemini, jadi HSV itu bukan untuk dilihat langsung oleh mata manusia, namun untuk mudah dibaca oleh komputer. pada visualisasi terlihat warna rgb sangat pekat. biasanya penggunaannya untuk mendeteksi objek berwarna misal deteksi jenis minuman di vending machine berdasarkan hsv gitu.

Soal 2 — Manipulasi Channel Warna RGB

- Gunakan gambar hasil crop dari Soal 1.
- Konversikan gambar ke ruang warna **RGB**.
- Lakukan manipulasi channel warna dengan cara:
 - **Naikkan intensitas channel merah sebanyak 50 poin** (maksimum 255).
 - **Turunkan intensitas channel biru sebanyak 30 poin** (minimum 0).
- Teknik atau cara menaikkan/menurunkan intensitas **dibebaskan**, asalkan logis dan hasilnya terlihat.
- Gabungkan kembali channel warna dan **simpan gambar hasil modifikasi dalam format .png**.
- **Tampilkan histogram per channel (R, G, B)** untuk gambar asli dan hasil modifikasi menggunakan `matplotlib.pyplot.hist`.
- Jelaskan dampak perubahan RGB pada warna gambar dalam sel **Markdown**.

```
# load gambar dari crop diatas
rgb_image = crop_wajah.copy()
print("Tinggi gambar:", rgb_image.shape[0], "piksel")
print("Lebar gambar:", rgb_image.shape[1], "piksel")
print("Jumlah channel:", rgb_image.shape[2], "(RGB)")

# Tampilkan gambar + histogram dalam 1 layout
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# Gambar asli
axs[0].imshow(rgb_image)
axs[0].set_title(f"Gambar Asli\nShape: {rgb_image.shape}", fontsize=11)
axs[0].axis('off')

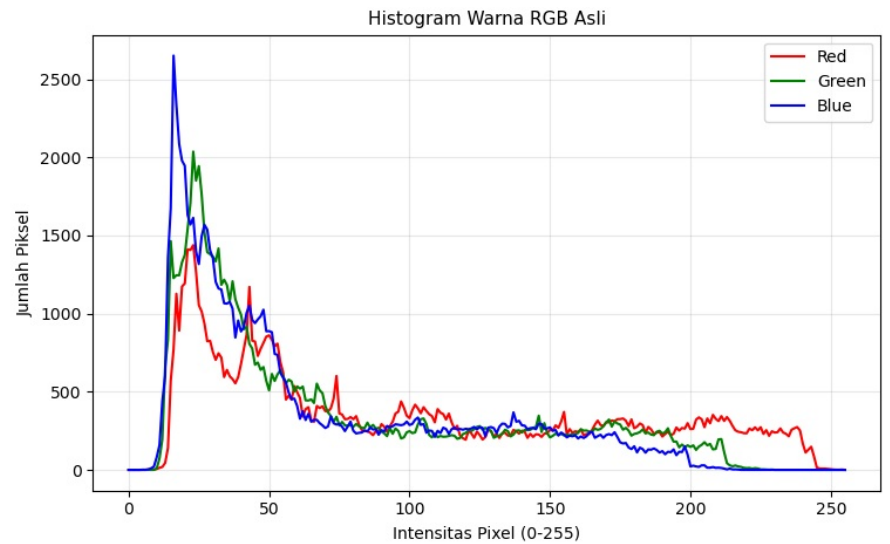
# Histogram RGB asli
colors = ('r', 'g', 'b')
labels = ('Red', 'Green', 'Blue')
for i, (col, label) in enumerate(zip(colors, labels)):
    hist = cv2.calcHist([rgb_image], [i], None, [256], [0, 256])
    axs[1].plot(hist, color=col, label=label)

axs[1].set_title("Histogram Warna RGB Asli", fontsize=11)
axs[1].set_xlabel("Intensitas Pixel (0-255)")
axs[1].set_ylabel("Jumlah Piksel")
axs[1].legend()
axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Tinggi gambar: 300 piksel
Lebar gambar: 300 piksel
Jumlah channel: 3 (RGB)

Gambar Asli
Shape: (300, 300, 3)



```
# Buat gambar modifikasi baru
img_mod_R = rgb_image.copy()

# --- 1. Naikkan intensitas channel Merah (R) sebanyak 50 ---
# Ambil channel Merah
r_channel = img_mod_R[:, :, 0]

# Buat matriks baru seukuran gambar, isinya 50
# .shape[:2] mengambil (tinggi, lebar)
matriks_50 = np.full(r_channel.shape, 50, dtype=np.uint8)

# Gunakan cv2.add() untuk menambah 50 dan otomatis clip di 255
r_channel_baru = cv2.add(r_channel, matriks_50)

# Masukkan kembali channel R yang sudah dimodifikasi
img_mod_R[:, :, 0] = r_channel_baru

# --- Tampilkan gambar dan histogramnya ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# Gambar hasil modifikasi
axs[0].imshow(img_mod_R)
axs[0].set_title("Hanya Merah +50", fontsize=11)
axs[0].axis('off')

# Histogram setelah modifikasi
colors = ('r', 'g', 'b')
labels = ('Merah (R)', 'Hijau (G)', 'Biru (B)')
for i, (col, label) in enumerate(zip(colors, labels)):
    hist = cv2.calcHist([img_mod_R], [i], None, [256], [0, 256])
    axs[1].plot(hist, color=col, label=label)

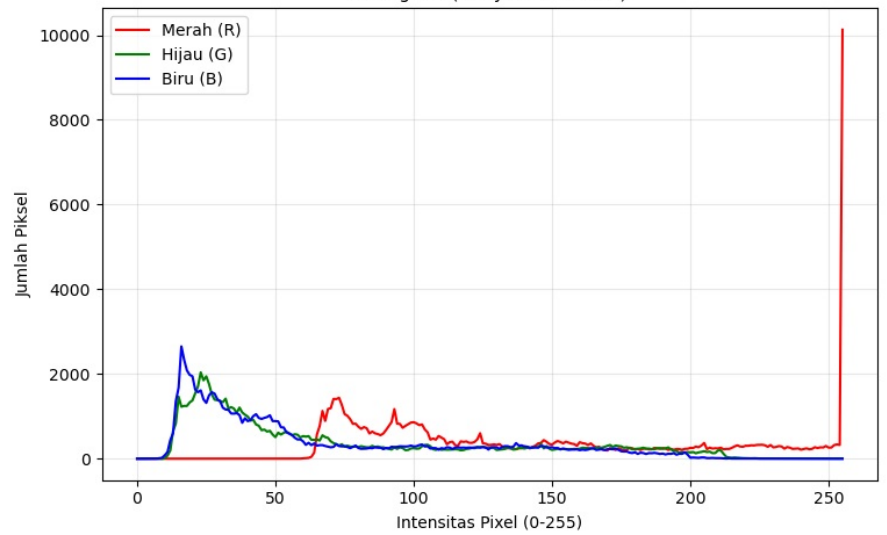
axs[1].set_title("Histogram (Hanya Merah +50)", fontsize=11)
axs[1].set_xlabel("Intensitas Pixel (0-255)")
axs[1].set_ylabel("Jumlah Piksel")
axs[1].legend()
axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```


Hanya Merah +50



Histogram (Hanya Merah +50)



```
# Buat gambar modifikasi HANYA UNTUK BIRU
img_mod_B = rgb_image.copy() # Ambil dari 'rgb_image' asli lagi

# --- Turunkan intensitas channel Biru (B) sebanyak 30 ---
b_channel = img_mod_B[:, :, 2]
matriks_30 = np.full(b_channel.shape, 30, dtype=np.uint8)
b_channel_baru = cv2.subtract(b_channel, matriks_30)
img_mod_B[:, :, 2] = b_channel_baru

# --- Tampilkan gambar dan histogramnya ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# Gambar hasil modifikasi
axs[0].imshow(img_mod_B)
axs[0].set_title("Hanya Biru -30", fontsize=11)
axs[0].axis('off')

# Histogram setelah modifikasi
for i, (col, label) in enumerate(zip(colors, labels)): # colors/labels dari sel sebelumnya
    hist = cv2.calcHist([img_mod_B], [i], None, [256], [0, 256])
    axs[1].plot(hist, color=col, label=label)

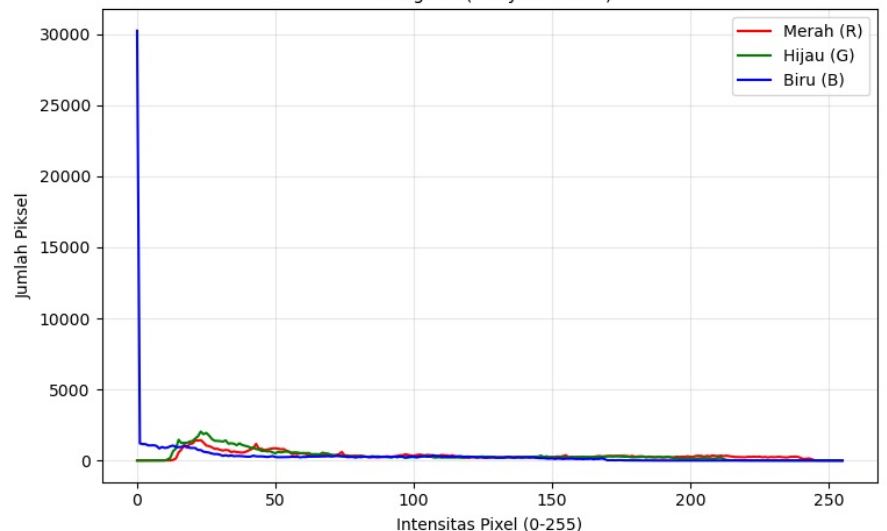
axs[1].set_title("Histogram (Hanya Biru -30)", fontsize=11)
axs[1].set_xlabel("Intensitas Pixel (0-255)")
axs[1].set_ylabel("Jumlah Pixel")
axs[1].legend()
axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Hanya Biru -30



Histogram (Hanya Biru -30)



```
# Buat gambar modifikasi GABUNGAN
img_mod_R_B = rgb_image.copy() # Ambil dari 'rgb_image' asli lagi

# --- 1. Naikkan Merah +50 ---
r_channel = img_mod_R_B[:, :, 0]
matriks_50 = np.full(r_channel.shape, 50, dtype=np.uint8)
r_channel_baru = cv2.add(r_channel, matriks_50)
```

```
img_mod_R_B[:, :, 0] = r_channel_baru

# --- 2. Turunkan Biru -30 ---
b_channel = img_mod_R_B[:, :, 2]
matriks_30 = np.full(b_channel.shape, 30, dtype=np.uint8)
b_channel_baru = cv2.subtract(b_channel, matriks_30)
img_mod_R_B[:, :, 2] = b_channel_baru

# --- 3. Tampilkan gambar dan histogramnya ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# Gambar hasil modifikasi
axs[0].imshow(img_mod_R_B)
axs[0].set_title("Gabungan: Merah +50, Biru -30", fontsize=11)
axs[0].axis('off')

# Histogram setelah modifikasi
for i, (col, label) in enumerate(zip(colors, labels)): # colors/labels dari sel sebelumnya
    hist = cv2.calcHist([img_mod_R_B], [i], None, [256], [0,256])
    axs[1].plot(hist, color=col, label=label)

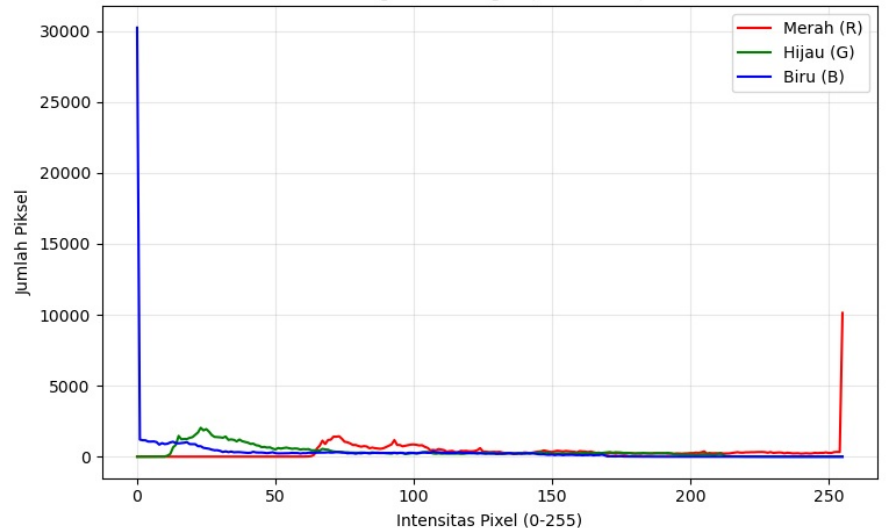
axs[1].set_title("Histogram Gabungan (R+50, B-30)", fontsize=11)
axs[1].set_xlabel("Intensitas Pixel (0-255)")
axs[1].set_ylabel("Jumlah Pixel")
axs[1].legend()
axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
# simpan hasil gambar gabungan
cv2.imwrite("results_ws4/gabungan_Rplus50_Bminus30.jpg", cv2.cvtColor(img_mod_R_B, cv2.COLOR_RGB2BGR))
```

Gabungan: Merah +50, Biru -30



Histogram Gabungan (R+50, B-30)



True

Penjelasan Dampak perubahan RGB pada warna gambar

- Pada gambar asli dapat dilihat sebaran histogramnya cukup merata mulai dari rentang 50-150. 50 kebawah intensitasnya cukup tinggi namun nilai intensitas di rentang bawah itu gelap.
- Pada gambar 2 perubahan rgb saat dinaikkan intensitas warna merah sebanyak 50, gambar seperti ada efek warm yang lumayan kontras warna merahnya. dan juga distribusi histogramnya sangat miring ke kanan artinya lebih dari 10k pixel itu intensitas nilai merahnya rentang angka 250.
- Pada gambar 3 perubahan rgb saat diturunkan intensitas warna biru sebanyak 30, entah kenapa cenderung terlihat berwarna kuning. seperti ada efek vintage gitu, karena dari histogramnya sangat miring ke kiri dimana lebih dari 30k pixel itu intensitas nilai birunya rentang angka 0.
- Sehingga pada gambar gabungan warna foto/gambarnya cenderung merah ada kuning"nya.

Soal 3 — Deteksi Tepi dan Filter Citra

- Ambil gambar **objek dengan background bertekstur** (misalnya kain bermotif, jerami, atau batu).
- Terapkan **edge detection (Canny)** dan tampilkan hasilnya.
- Lakukan **thresholding dengan nilai ambang tertentu** (bebas Anda tentukan) agar hanya objek utama yang tersisa.
- Buat **bounding box** di sekitar objek hasil segmentasi (boleh manual atau otomatis).
- Terapkan **filter blur** dan **filter sharpening**, lalu **bandingkan hasil keduanya**.
- Jelaskan bagaimana setiap filter memengaruhi detail gambar dalam format **Markdown**.

```
image_laptop_path = 'assets_ws4/edges.jpeg' # Asumsi nama file adalah 'laptop.jpg'
```



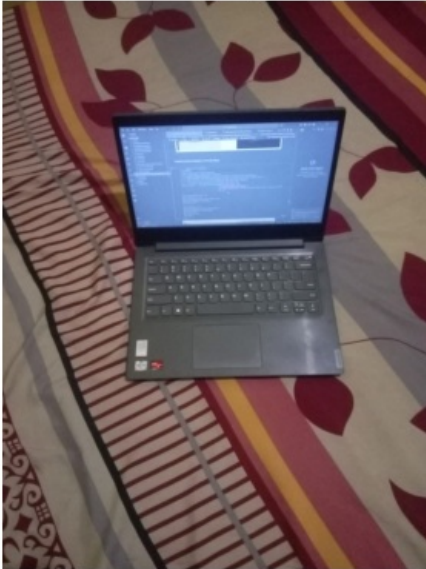
```
# Baca gambar
image_laptop_bgr = cv2.imread(image_laptop_path)

# Konversi dari BGR (OpenCV) ke RGB (Matplotlib)
image_laptop_rgb = cv2.cvtColor(image_laptop_bgr, cv2.COLOR_BGR2RGB)

plt.imshow(image_laptop_rgb)
plt.title('Original Laptop Image')
plt.axis('off')
plt.show()

# Simpan versi grayscale untuk beberapa operasi selanjutnya
image_laptop_gray = cv2.cvtColor(image_laptop_bgr, cv2.COLOR_BGR2GRAY)
```

Original Laptop Image



Segmentation

Implementasi Canny Edge Detection

```
# --- Canny Edge Detection ---
# threshold1 & threshold2: thresholding hysteresis
low_threshold = 70
high_threshold = 230
canny_edges = cv2.Canny(image_laptop_gray, low_threshold, high_threshold)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(image_laptop_gray, cmap='gray')
plt.title('Original Grayscale')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(canny_edges, cmap='gray')
plt.title(f'Canny Edges (Low:{low_threshold}, High:{high_threshold})')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Original Grayscale



Canny Edges (Low:70, High:230)



Implementasi Adaptive Thresholding

```
# --- Adaptive Thresholding ---
# Parameter:
# image_laptop_gray: input grayscale
# 255: nilai piksel jika di atas threshold
# cv2.ADAPTIVE_THRESH_GAUSSIAN_C: metode perhitungan threshold (bisa juga MEAN_C)
# cv2.THRESH_BINARY: tipe threshold (piksel di atas threshold jadi 255, di bawah jadi 0)
# 11: block size (ukuran area untuk menghitung threshold, harus ganjil)
# 2: C (konstanta yang dikurangi dari mean/gaussian rata-rata)
adaptive_thresh = cv2.adaptiveThreshold(image_laptop_gray, 255,
                                       cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                       cv2.THRESH_BINARY_INV, # Pakai INV agar objek putih, background hitam
                                       13, 2)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(image_laptop_gray, cmap='gray')
plt.title('Original Grayscale')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(adaptive_thresh, cmap='gray')
plt.title('Adaptive Thresholding')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Original Grayscale



Adaptive Thresholding



```
# --- Bounding Box setelah Adaptive Thresholding ---
```

```
# 1. Cari kontur pada gambar hasil adaptive threshold
# cv2.RETR_EXTERNAL: hanya mencari kontur terluar
# cv2.CHAIN_APPROX_SIMPLE: menyimpan hanya titik-titik esensial dari kontur
contours, _ = cv2.findContours(adaptive_thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
# Buat salinan gambar asli untuk digambar bounding box
image_with_bbox = image_laptop_rgb.copy()

# Inisialisasi variabel untuk menyimpan bounding box terbesar (diasumsikan laptop adalah objek terbesar)
largest_contour = None
max_area = 0

# Iterasi melalui semua kontur yang ditemukan
for contour in contours:
    area = cv2.contourArea(contour)
    # Filter kontur yang terlalu kecil (bisa jadi noise)
    if area > 1000: # Anda mungkin perlu menyesuaikan nilai threshold area ini
        if area > max_area:
            max_area = area
            largest_contour = contour

# Jika kontur terbesar ditemukan, gambar bounding box-nya
if largest_contour is not None:
    # Mendapatkan koordinat (x, y, lebar, tinggi) dari bounding box
    x, y, w, h = cv2.boundingRect(largest_contour)
    # Gambar persegi panjang pada gambar asli
    # (x, y) adalah pojok kiri atas
    # (x+w, y+h) adalah pojok kanan bawah
    # (255, 0, 0) adalah warna Merah dalam RGB
    # 2 adalah ketebalan garis
    cv2.rectangle(image_with_bbox, (x, y), (x + w, y + h), (255, 0, 0), 2)
    # Opsional: tambahkan teks label
    cv2.putText(image_with_bbox, 'Laptop', (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255,0,0), 2)
else:
    print("Tidak ada kontur yang cukup besar ditemukan untuk bounding box.")

plt.figure(figsize=(6, 6))
plt.imshow(image_with_bbox)
plt.title('Laptop dengan Bounding Box (dari Adaptive Threshold)')
plt.axis('off')
plt.show()
```

Laptop dengan Bounding Box (dari Adaptive Threshold)



Analisis Segmentasi dan Labeling bounding box.

- disini pakai canny karna sejauh yang saya tahu edge detection yang sangat bagus adalah canny. pada canny saya atur low threshold di 70 dan hgh threshold di 230. karena motif bunga sangat mengganggu jadi saya fokus hilangkan itu aja, dan juga motif garis"itu sangat kuat sehingga tidak bisa saya fokus hanya ke laptop tapi setidaknya bentuk tepi laptop masih terlihat.
- selanjutnya untuk thresholding saya pakai adaptive daripada otsu, karena otsu ngasi threshold secara global, sedangkan saya mau atur block size dan konstantanya, sejauh yang saya coba block size 13 lebih bagus untuk nantinya di bounding box, dibanding block size 11 yang masih agak lebar ke kanan.

Filtering

Gaussian Blur

```
# 1. Gaussian Blur
# (ksize) harus angka ganjil, misal (7, 7)
ksize_gaussian = (11, 11)
gaussian_blur = cv2.GaussianBlur(image_laptop_rgb, ksize_gaussian, 0)

# Tampilkan perbandingan
plt.figure(figsize=(9, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_laptop_rgb)
plt.title('Original')
plt.axis('off')

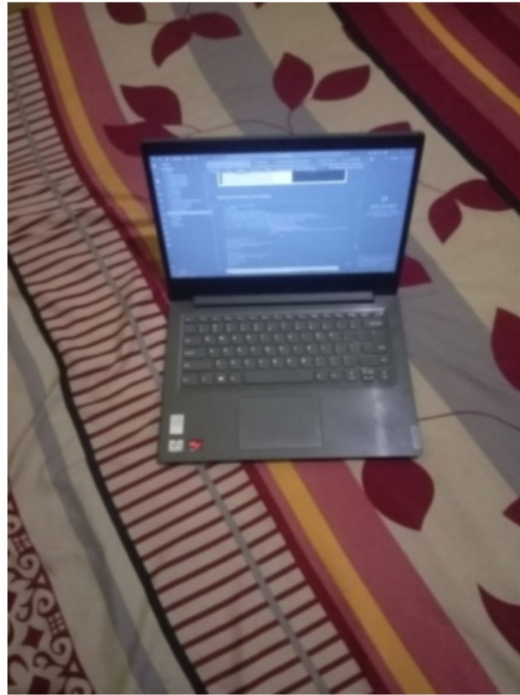
plt.subplot(1, 2, 2)
plt.imshow(gaussian_blur)
plt.title(f'Gaussian Blur (Kernel {ksize_gaussian})')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original



Gaussian Blur (Kernel (11, 11))



```
# 1. Buat Kernel Sharpening
# Angka 9 di tengah "meningkatkan" piksel pusat
# Angka -1 di sekelilingnya "mengurangi" nilai tetangganya
# Total bobot kernel (9 + 8*-1) = 1, sehingga kecerahan gambar tetap
kernel_sharpen = np.array([[-1, -1, -1],
                             [-1, 9, -1],
                             [-1, -1, -1]])

# 2. Terapkan kernel menggunakan cv2.filter2D()
# '-1' berarti kedalaman (tipe data) output sama dengan input
sharpened_image = cv2.filter2D(image_laptop_rgb, -1, kernel_sharpen)

# Tampilkan perbandingan
plt.figure(figsize=(10, 6))

plt.subplot(1, 2, 1)
plt.imshow(image_laptop_rgb)
plt.title('Original')
plt.axis('off')

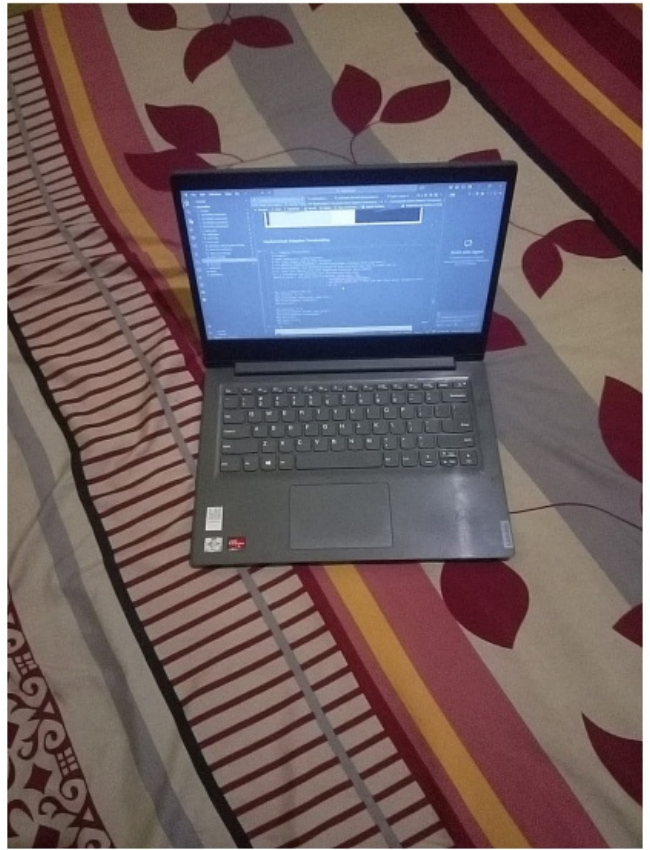
plt.subplot(1, 2, 2)
plt.imshow(sharpened_image)
plt.title('Sharpened Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```


Original



Sharpened Image



```
# Visualisasi gaussian blur dan sharpening pada gambar laptop
plt.figure(figsize=(10, 5))

plt.subplot(1, 3, 1)
plt.imshow(image_laptop_rgb)
plt.title('Gambar Asli')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(gaussian_blur)
plt.title('Gaussian Blur')
plt.axis('off')

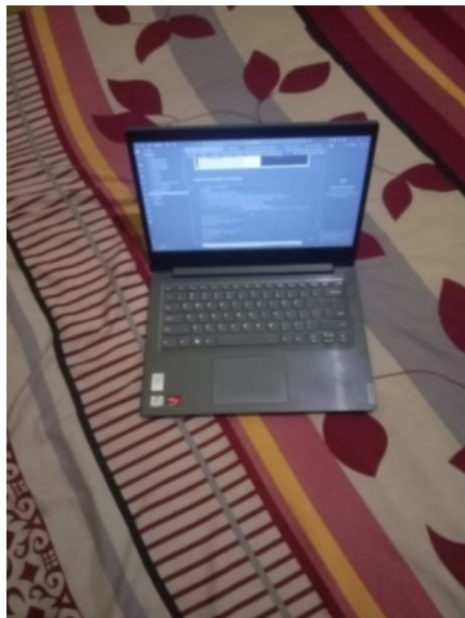
plt.subplot(1, 3, 3)
plt.imshow(sharpened_image)
plt.title('Sharpened Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

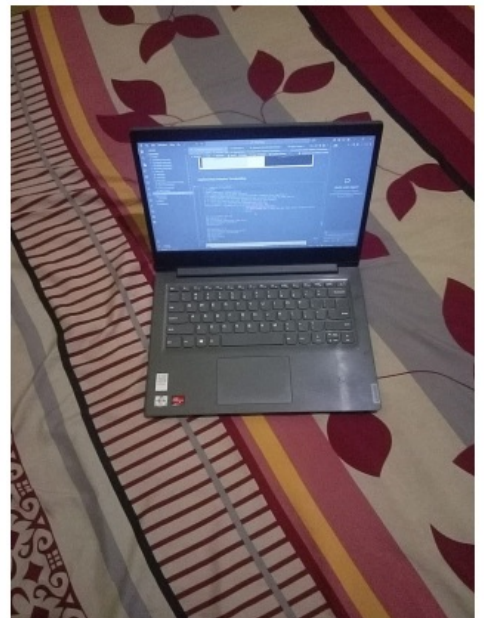
Gambar Asli



Gaussian Blur



Sharpened Image



Analisis perbandingan gaussian dan sharpening filtering

- Gaussian blur. lebih blur dibanding gambar asli tentu saja, biasanya dipakai untuk preprocessing agar tidak banyak noise/ngurangi noise. bisa dilihat background nya terlihat lebih halus teksturnya, namun foreground nya blur.
- Sharpening dibanding gambar asli dari kamera saya jauh lebih jelas dan tajam, tepinya lebih jelas terlihat, namun tekstur dan noise nya juga jadi lebih jelas.

Soal 4 — Deteksi Wajah dan Filter Digital Kreatif

- Ambil gambar diri Anda dengan ekspresi wajah **netral**.
- Lakukan **deteksi wajah dan landmark** menggunakan salah satu dari:
 - **MediaPipe**, atau
 - **Dlib**, atau
 - **OpenCV**.
- Buat **overlay filter digital kreatif** karya Anda sendiri, misalnya:
 - topi, kumis, masker, helm, aksesoris, atau bentuk unik lainnya.
 - Filter boleh dibuat dari **gambar eksternal (PNG)** *atau* digambar langsung (misal bentuk lingkaran, garis, poligon, dll).
- Pastikan posisi overlay menyesuaikan **landmark wajah** dengan logis.
- **Gunakan alpha blending sebagai saran** agar hasil tampak lebih natural.
- Tampilkan perbandingan antara **gambar asli** dan **hasil dengan filter**.
- Jelaskan bagaimana Anda menghitung posisi overlay dan tantangan yang dihadapi selama implementasi (gunakan **Markdown**).

```
# --- Tentukan Path Gambar ---
image_me_path = 'assets_ws4/me.jpeg'
image_crown_path = 'assets_ws4/mugiwara.png'

# Inisialisasi variabel
img_me_rgb = None
img_crown_rgba = None

# Baca gambar wajah
if not os.path.exists(image_me_path):
    print(f"Error: Tidak ditemukan file '{image_me_path}'")
else:
    img_me_bgr = cv2.imread(image_me_path)
    img_me_rgb = cv2.cvtColor(img_me_bgr, cv2.COLOR_BGR2RGB)

# Baca gambar mahkota
if not os.path.exists(image_crown_path):
    print(f"Error: Tidak ditemukan file '{image_crown_path}'")
else:
    img_crown_bgra = cv2.imread(image_crown_path, cv2.IMREAD_UNCHANGED)
    img_crown_rgba = cv2.cvtColor(img_crown_bgra, cv2.COLOR_BGRA2RGBA)

plt.figure(figsize=(6, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_me_rgb)
plt.title("Gambar saya")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(img_crown_rgba)
plt.title("Gambar Filter")
plt.axis('off')

plt.tight_layout() # Merapikan jarak
plt.show()
```


Gambar saya



Gambar Filter



```
# 1. Inisialisasi MediaPipe Face Mesh
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
drawing_spec = mp_drawing.DrawingSpec(thickness=1, circle_radius=1, color=(0, 255, 0))

# 2. Siapkan gambar salinan untuk anotasi
annotated_image = img_me_rgb.copy()

# 3. Jalankan deteksi
with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    # --- MODIFIKASI: Turunkan confidence agar lebih mudah deteksi ---
    min_detection_confidence=0.2) as face_mesh: # <--- Diubah dari 0.5 ke 0.2

    results = face_mesh.process(annotated_image)

# 4. Gambar landmark jika wajah terdeteksi
if not results.multi_face_landmarks:
    print("Error: Wajah tetap tidak terdeteksi, bahkan dengan confidence 0.2.")
    print("SOLUSI: Harap ganti foto 'me.jpeg' Anda dengan foto selfie yang lebih jelas.")
else:
    face_landmarks = results.multi_face_landmarks[0]

    # Gambar mesh di wajah
    mp_drawing.draw_landmarks(
        image=annotated_image,
        landmark_list=face_landmarks,
        connections=mp_face_mesh.FACEMESH_TESSELATION,
        landmark_drawing_spec=drawing_spec,
        connection_drawing_spec=drawing_spec)

# 5. Tampilkan hasil landmark
plt.figure(figsize=(4, 4))
plt.imshow(annotated_image)
plt.title("skeleton face Landmark")
plt.axis('off')
plt.show()
```

skeleton face Landmark



```

# --- 6. Hitung Posisi & Ukuran Mahkota ---
# Kita asumsikan 'face_landmarks' sudah ada dari sel sebelumnya

# Dapatkan tinggi dan lebar gambar asli
h, w, _ = img_me_rgb.shape

# Kita gunakan landmark di pelipis kiri (130) dan kanan (359)
# (Ini mungkin perlu disesuaikan, misal 234 dan 454)
left_temple = face_landmarks.landmark[234]
right_temple = face_landmarks.landmark[454]

# Hitung lebar wajah dalam piksel
face_width_pixels = int((right_temple.x - left_temple.x) * w)

# Atur lebar mahkota (misal, 1.5x lebih lebar dari jarak pelipis)
crown_target_width = int(face_width_pixels * 2.5)

# B. Resize Mahkota
# Asumsi 'img_crown_rgba' sudah ada dari sel load dataset
crown_h, crown_w, _ = img_crown_rgba.shape
aspect_ratio = crown_h / crown_w
crown_resized_h = int(crown_target_width * aspect_ratio)
crown_resized_w = crown_target_width

# Resize gambar mahkota
img_crown_resized = cv2.resize(img_crown_rgba, (crown_resized_w, crown_resized_h),
                               interpolation=cv2.INTER_AREA)

# C. Tentukan Posisi (x, y) untuk Menempelkan
# Kita gunakan landmark 10 (tengah dahi atas) sebagai 'jangkar'
top_forehead = face_landmarks.landmark[10]
forehead_x = int(top_forehead.x * w)
forehead_y = int(top_forehead.y * h)

# Hitung posisi x,y (pojok kiri atas) dari mahkota
# Posisikan mahkota agar 'tengah-bawah'-nya ada di dahi
# Kita geser ke atas sebesar 80% dari tinggi mahkota (offset)
pos_y = forehead_y - int(crown_resized_h * 0.7)
# Kita geser ke kiri sebesar 50% dari lebar mahkota (pusatkan)
pos_x = forehead_x - int(crown_resized_w / 2)

# Debug: cek kalkulasi
print(f"Lebar wajah (pelipis): {face_width_pixels}px")
print(f"Target lebar mahkota: {crown_target_width}px")
print(f"Titik jangkar dahi (x, y): ({forehead_x}, {forehead_y})")
print(f"Posisi akhir mahkota (x, y): ({pos_x}, {pos_y})")

```

```

Lebar wajah (pelipis): 292px
Target lebar mahkota: 730px
Titik jangkar dahi (x, y): (455, 252)
Posisi akhir mahkota (x, y): (90, -80)

```

```

# --- 7. Terapkan Alpha Blending (Versi Perbaikan) ---
# Buat salinan gambar asli untuk ditimpa
final_image = img_me_rgb.copy()

# Dapatkan ukuran gambar mahkota yang sudah di-resize
c_h, c_w, _ = img_crown_resized.shape

# Tentukan Region of Interest (ROI) di gambar utama
y1 = pos_y                # Ini bisa jadi negatif (misal: -15)
y2 = pos_y + c_h          # e.g., -15 + 100 = 85
x1 = pos_x
x2 = pos_x + c_w

# --- Koreksi Batas BARU (Lebih Cerdas) ---
# Kita perlu tahu berapa banyak mahkota yang harus di-slice dari ATAS dan KIRI

# Tentukan koordinat slice untuk crown (awalnya, potong 0)
c_y1, c_x1 = 0, 0

# Dapatkan tinggi/lebar gambar utama
h, w, _ = final_image.shape

# Koreksi ATAS (Y1)
if y1 < 0:
    c_y1 = abs(y1)         # Kita harus mulai slice crown dari (e.g., 15)
    c_h = c_h - c_y1       # Tinggi crown baru adalah (e.g., 100 - 15 = 85)
    y1 = 0                 # Posisi tempel di gambar utama dimulai dari 0

# Koreksi KIRI (X1)
if x1 < 0:
    c_x1 = abs(x1)         # Kita harus mulai slice crown dari (e.g., 10)

```

```

c_w = c_w - c_x1      # Lebar crown baru
x1 = 0

# Koreksi BAWAH (Y2)
if y2 > h:
    c_h = c_h - (y2 - h) # Kurangi tinggi crown yang terpotong di bawah
    y2 = h               # Batasi y2 ke tinggi maks gambar

# Koreksi KANAN (X2)
if x2 > w:
    c_w = c_w - (x2 - w) # Kurangi lebar crown yang terpotong di kanan
    x2 = w

# --- Ambil Potongan Mahkota yang SUDAH DIKOREKSI ---
# Potong mahkota berdasarkan koordinat slice c_y1 dan c_x1
img_crown_final = img_crown_resized[c_y1 : c_y1 + c_h, c_x1 : c_x1 + c_w]

# Ambil ROI dari gambar final (dengan koordinat yang sudah dikoreksi)
roi = final_image[y1:y2, x1:x2]

# --- Mulai Alpha Blending ---
# (Safety check: jika ukuran masih beda lpx, paksaan resize)
if roi.shape[0] != img_crown_final.shape[0] or roi.shape[1] != img_crown_final.shape[1]:
    img_crown_final = cv2.resize(img_crown_final, (roi.shape[1], roi.shape[0]))

# Pisahkan RGB dan Alpha dari mahkota (yang sudah RGBA)
crown_rgb = img_crown_final[:, :, :3]
crown_alpha = img_crown_final[:, :, 3]

# Normalisasi alpha channel (0-255 menjadi 0.0-1.0)
alpha = crown_alpha / 255.0

# Buat mask 3 channel dari alpha
alpha_mask = cv2.merge([alpha, alpha, alpha])

# Terapkan rumus blending: (FG * alpha) + (BG * (1 - alpha))
blended_roi = (alpha_mask * crown_rgb) + ((1 - alpha_mask) * roi)

# Konversi kembali ke tipe data gambar (uint8)
blended_roi = blended_roi.astype(np.uint8)

# Tempelkan ROI yang sudah di-blend kembali ke gambar utama
final_image[y1:y2, x1:x2] = blended_roi

# --- 8. Tampilkan Hasil Akhir ---
plt.figure(figsize=(4, 4))
plt.imshow(final_image)
plt.title("Selfie dengan Mahkota!")
plt.axis('off')
plt.show()

```

Selfie dengan Mahkota!



```

# visualisasikan perbandingan sebelum dan sesudah pemasangan mahkota
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_me_rgb)
plt.title("Sebelum Mahkota")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(final_image)
plt.title("Sesudah Mahkota")

```

```
plt.axis('off')

plt.tight_layout()
plt.show()
```

Sebelum Mahkota



Sesudah Mahkota



Analisis overlay dan tantangan

- Overlay dari skala dengan hitung landmark namun saya ga ketemu dokumentasinya jadi saya coba beberapa saran dari ai untuk indeks landmark nyocokkin di bagian pelipis. Akhirnya, saya pakai landmark 234 dan 454 buat ngukur lebar dahi. Lebar mahkotanya saya atur jadi 2.5x lipat dari lebar dahi itu biar kelihatan pas
- tantangannya adalah nyocokkin skala, posisi, dan lebarnya. namun untuk posisi dan skala cukup sulit dan saya rasa masih kurang maksimal hasilnya.

Soal 5 — Perspektif dan Peningkatan Kualitas Citra

- Ambil gambar **objek datar** seperti karya tangan di kertas, tulisan di papan tulis, atau foto produk di meja dengan kondisi pencahayaan atau sudut yang tidak ideal.
- Lakukan **preprocessing** untuk memperbaiki tampilan agar lebih rapi dan jelas, dengan langkah-langkah:
 - Konversi ke **grayscale**.
 - **Koreksi perspektif (transformasi homografi)** menggunakan **4 titik manual** agar objek terlihat sejajar dan tidak terdistorsi.
 - Terapkan **thresholding adaptif atau Otsu** (pilih salah satu, dan jelaskan alasan pilihan Anda).
- Tampilkan **setiap tahap pemrosesan dalam satu grid** agar mudah dibandingkan.
- Jelaskan fungsi masing-masing tahap dan bagaimana teknik ini meningkatkan kualitas visual citra (gunakan **Markdown**).

```
image_path = 'assets_ws4/art.jpeg'

# 1. Load Gambar
if not os.path.exists(image_path):
    print(f"Error: Tidak ditemukan file '{image_path}'")
else:
    # Baca gambar asli (BBR)
    img_original_bgr = cv2.imread(image_path)

    # --- TAHAP 1: Konversi ke Grayscale ---
    img_gray = cv2.cvtColor(img_original_bgr, cv2.COLOR_BGR2GRAY)

    # Tampilkan perbandingan
    plt.figure(figsize=(8, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(img_original_bgr, cv2.COLOR_BGR2RGB)) # Konversi ke RGB untuk plt
    plt.title("1. Gambar Asli (RGB)")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(img_gray, cmap='gray') # Tampilkan grayscale
    plt.title("2. Hasil Tahap 1 (Grayscale)")
    plt.axis('off')

    plt.tight_layout()
    plt.show()
```

```
print(f"Ukuran gambar asli: {img_original_bgr.shape}")
print(f"Ukuran gambar grayscale: {img_gray.shape}")
```

1. Gambar Asli (RGB)



2. Hasil Tahap 1 (Grayscale)



Ukuran gambar asli: (1040, 780, 3)
Ukuran gambar grayscale: (1040, 780)

```
pts_src = np.array([
    [153, 200], # Kiri Atas
    [621, 235], # Kanan Atas
    [104, 939], # Kiri Bawah
    [671, 935] # Kanan Bawah
], dtype=np.float32)

# Ambil lebar terpanjang (antara atas atau bawah)
width_top = np.sqrt(((pts_src[1][0] - pts_src[0][0])**2) + ((pts_src[1][1] - pts_src[0][1])**2))
width_bottom = np.sqrt(((pts_src[3][0] - pts_src[2][0])**2) + ((pts_src[3][1] - pts_src[2][1])**2))
max_width = int(max(width_top, width_bottom))

# Ambil tinggi terpanjang (antara kiri atau kanan)
height_left = np.sqrt(((pts_src[2][0] - pts_src[0][0])**2) + ((pts_src[2][1] - pts_src[0][1])**2))
height_right = np.sqrt(((pts_src[3][0] - pts_src[1][0])**2) + ((pts_src[3][1] - pts_src[1][1])**2))
max_height = int(max(height_left, height_right))

print(f"Ukuran target 'bird's-eye view': {max_width}w x {max_height}h")

# 3. Definisikan 4 Titik TUJUAN (Destination)
# Ini adalah persegi panjang lurus yang kita inginkan
# (0, 0)
# (lebar, 0)
# (0, tinggi)
# (lebar, tinggi)
pts_dst = np.array([
    [0, 0], # Kiri Atas
    [max_width - 1, 0], # Kanan Atas
    [0, max_height - 1], # Kiri Bawah
    [max_width - 1, max_height - 1] # Kanan Bawah
], dtype=np.float32)

# 4. Hitung Matriks Transformasi Perspektif
# Ini adalah "kunci" yang memberi tahu OpenCV cara memetakan pts_src ke pts_dst
matrix = cv2.getPerspectiveTransform(pts_src, pts_dst)

# 5. Terapkan Transformasi (Warping)
# Kita gunakan 'img_gray' dari sel sebelumnya
# (max_width, max_height) adalah ukuran gambar output
img_warped = cv2.warpPerspective(img_gray, matrix, (max_width, max_height))

# 6. Tampilkan Hasil
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_gray, cmap='gray')
plt.title("Gambar Asli (Grayscale)")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(img_warped, cmap='gray')
plt.title("Hasil Perspective Transform\n(Bird's-Eye View)")
plt.axis('off')

plt.tight_layout()
```



```
plt.show()
```

Ukuran target 'bird's-eye view': 567w x 740h

Gambar Asli (Grayscale)



Hasil Perspective Transform
(Bird's-Eye View)



```
# Pastikan 'img_warped' sudah ada dari Tahap 3
if 'img_warped' in locals():

    # --- TAHAP 4: Terapkan Adaptive Thresholding ---
    img_adaptive_thresh = cv2.adaptiveThreshold(
        img_warped,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        11,
        2
    )

    # Tampilkan hasil
    plt.figure(figsize=(7, 7))
    plt.imshow(img_adaptive_thresh, cmap='gray')
    plt.title("Hasil Tahap 4: Adaptive Thresholding")
    plt.axis('off')
    plt.show()

else:
    print("Error: 'img_warped' tidak ditemukan.")
    print("Pastikan Tahap 3 (Koreksi Perspektif) sudah dijalankan.")
```

Hasil Tahap 4: Adaptive Thresholding



```
# visualisasi semua gambar hasil tiap tahap
plt.figure(figsize=(15, 5))

plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(img_original_bgr, cv2.COLOR_BGR2RGB))
plt.title("1. Gambar Asli (RGB)")
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(img_gray, cmap='gray')
plt.title("2. Hasil Grayscale")
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(img_warped, cmap='gray')
plt.title("3. Hasil Koreksi Perspektif")
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(img_adaptive_thresh, cmap='gray')
plt.title("4. Hasil Adaptive Thresholding")
plt.axis('off')

plt.tight_layout()
plt.show()
```

1. Gambar Asli (RGB)



2. Hasil Grayscale



3. Hasil Koreksi Perspektif



4. Hasil Adaptive Thresholding



- jadi tahap pertama adalah load data, dan kenapa dijadiin greyscale? agar lebih mudah untuk mengenali objek atau bentuknya karena dia juga kan 1 channel jadi lebih sederhana juga dibaca komputer. kemudian 4 titik koordinat untuk sudutnya jadi dengan cv2.warpPerspective akan ditarik gambar yang agak terdistorsi tadi jadi lurus sejajar seperti cam scanner. saya pilih adaptive daripada otsu karena otsu ngasih ambang batas secara global, sedangkan adaptive lokal. jadi adaptive scenderung cocok untuk mengatasi pencahayaan yang tidak rata.

Aturan Umum Pengerjaan

- Kerjakan secara **mandiri**.
- Bantuan AI (seperti ChatGPT, Copilot, dsb.) diperbolehkan **dengan bukti percakapan** (screenshot / link / script percakapan).
- Source code antar mahasiswa harus berbeda.
- Jika mendapat bantuan teman, tuliskan nama dan NIM teman yang membantu.
- Plagiarisme akan dikenakan sanksi sesuai aturan akademik ITERA.
- Cantumkan seluruh **credit dan referensi** yang digunakan di bagian akhir notebook.
- Penjelasan setiap soal ditulis dalam **Markdown**, bukan di dalam komentar kode.

Aturan Pengumpulan

- Semua file kerja Anda (notebook `.ipynb`, gambar, dan hasil) **wajib diunggah ke GitHub repository tugas sebelumnya**.

- Gunakan struktur folder berikut di dalam repo Anda:

```
/Nama_NIM_Repo/ # Nama repo sebelumnya
├── assets_ws4/   # berisi semua gambar atau video asli (input)
├── results_ws4/  # berisi semua hasil modifikasi dan output
├── worksheet4.ipynb
└── NIM_Worksheet4.pdf
```

- File yang dikumpulkan ke **Tally** hanya berupa **hasil PDF** dari notebook Anda, dengan format nama:

```
NIM_Worksheet4.pdf
```

- Pastikan notebook telah dijalankan penuh sebelum diekspor ke PDF.
- Sertakan tautan ke repository GitHub Anda di bagian atas notebook atau di halaman pertama PDF.

✔ Catatan Akhir

Worksheet 4 ini bertujuan mengasah pemahaman Anda tentang manipulasi citra digital secara praktis. Gunakan kreativitas Anda untuk menghasilkan hasil visual yang menarik dan penjelasan konseptual yang jelas.